☆ C++ Complete Notes ☆

C++

• From Basic to Advanced •

Learn • Practice • Master

☆ Powerful
High performance general-purpose programming language.

☆ Portable
Write once, compile and run on different platforms.



☆ Flexible
Supports multiple paradigms – Procedural, OOP, Generic.

☆ Efficient
Optimized for speed and memory management.

> "C++ is not just a language, it's a power that builds possibilities."

What You Will Learn

- Fundamentals
- Object-Oriented Programming
- STL (Standard Template Library)
- File Handling
- Advanced Concepts
- Practical Examples & Projects

Perfect For

- Beginners
- Students
- Placement Preparation
- Interview Preparation
- Competitive Programming
- Real World Applications

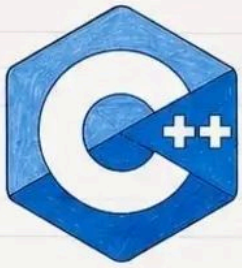
★ Skills and Career Path in Software ★

1. What is C++ ?

C++ is a general-purpose, high-level programming language.

It was developed as an extension of C language.

It supports multiple programming paradigms including Procedural, Object-Oriented and Generic programming.



C++ is not just a language, it's a powerful tool to build amazing things.

2. Origin and Developer



- → C++ was developed by Bjarne Stroustrup.
- → Development began in 1979 at Bell Labs.
- → It was initially called "C with Classes".
- → The name was later changed to C++ in 1983.

3. Why was C++ created ?

The main goal was to add Object-Oriented features to C language while maintaining efficiency and flexibility.

Problems in C		Solutions in C++
• No support for classes and objects. • Limited reusability of code. • Not suitable for large scale projects.	C++ →	• Added OOP concepts (Classes, Objects). • Code reusability through inheritance. • Suitable for real-world large projects.

4. Key Points about C++

- ★ High performance and efficiency.
- ★ Middle-level language (combination of low-level and high-level features).
- ★ Supports both procedural and object-oriented programming.
- ★ Portable – can run on multiple platforms.
- ★ Rich standard library (STL).
- ★ Widely used in industry, games, embedded systems, and more.



C++ is a foundation of many modern programming languages like Java, C#, Python and many more.

5. Features of C++

① Simple :- Easy to learn and understand.

② Machine Independent :- Can run on different platforms with little or no modification.

③ Object-Oriented :- Supports classes, objects, inheritance, polymorphism etc.

④ Rich Library :- Large collection of built-in functions and STL.

⑤ High Performance :- Fast and efficient compared to many other languages.

⑥ Extensible :- New features can be added easily.

⑦ Mid-level Language :- Combination of both high-level and low-level language features.

Did You Know?

C++ was designed with a goal of "Zero-overhead Abstraction", which means you get powerful features without sacrificing performance.

Remember!

C is a subset of C++. Every C program is a valid C++ program, but every C++ program is not a C program.

6. C++ vs Other Languages

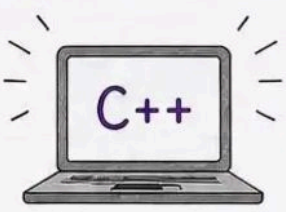
Feature	C	C++	Java	Python
Type	Procedural	Multi-paradigm (OOP + Procedural + Generic)	Object-Oriented	High-level
Performance	High	Very High	Medium (JVM based)	Lower (Interpreted)
Memory Control	Manual	Manual (with features)	Automatic (Garbage Collection)	Automatic (Garbage Collection)
Platform Dependency	Machine Dependent	Machine Independent	Machine Independent	Machine Independent
Compilation	Compiled	Compiled	Compiled + Interpreted (JVM)	Interpreted
Best Use	System Programming	System Programming, Game Dev, App Dev	Enterprise Applications	Scripting, AI, Data Science

7. Applications of C++

- Operating Systems
- Game Development
- Embedded Systems
- Compilers & Interpreters
- Database Systems

Examples

→ Windows OS, Linux Kernel
→ Unreal Engine, Unity, 3D Games
→ IoT devices, Microcontrollers
→ GCC Compiler, Clang
→ MySQL, MongoDB



8. Structure of a C++ Program

```
#include <iostream>          // Preprocessor Directive

using namespace std;         // Namespace

int main() {                 // Main Function

    // Statements

    cout << "Hello, World!" << endl;

    return 0;

}                            // End of Function
```

1. Preprocessor Directive
Commands that are processed before compilation.

2. Namespace
Defines the scope of identifiers. (std is the standard namespace)

3. Main Function
Every C++ program must have main() function. It is the entry point of the program.

4. Statements
Instructions written inside {} are executed by the program.

5. return 0;
Indicates successful execution of the program.

9. How C++ Program is Executed ?

1. Write Code
Write the C++ source code in .cpp file.

2. Preprocess
Preprocessor processes the code (macros, includes, etc.).

3. Compile
Compiler checks syntax and converts code to object code.

4. Link
Linker links object files with libraries to create executable file.

5. Execute
Executable file is run and produces output.

10. First C++ Program

```
#include <iostream>
using namespace std;
int main() {
    cout << "Hello, World!" << endl;
    return 0;
}
```

Output :

```
Hello, World!
```

Explanation :

- cout is used to display output.
- "Hello, World!" is the message.
- endl moves to the next line.
- return 0; means program executed successfully.

Note :
Every C++ program must have main() function. Without main(), program execution will not start.

11. Basic Syntax Explanation

```
#include <iostream>          // Preprocessor Directive
using namespace std;         // Namespace
int main() {                 // Main Function
    cout << "Hello, World!" << endl;  // Statement
    return 0;                // Return Statement
}                            // End of Function
```

1. Preprocessor Directive
It tells the compiler to include a header file (iostream in this case) before compilation.

2. Namespace
It avoids writing std:: before every standard library function.

3. Main Function
Every C++ program execution starts from main() function.

4. Statement
Instructions are the actual commands executed by the program.

5. Return Statement
return 0; indicates successful execution of the program.

12. Comments in C++

Type	Syntax	Example
Single Line Comment	// comment	// This is a single line comment
Multi Line Comment	/* comment */	/* This is a multi line comment */

Comments are ignored by the compiler. They are used to explain the code.

13. Output of the First Program

Code:

```
cout << "Hello, World!" << endl;
```

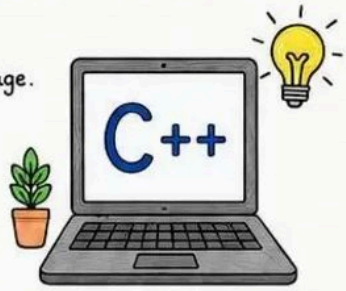
Output (on screen):

```
Hello, World!
```

Note:
endl is used to move the cursor to the next line and flush the output buffer.

14. Key Takeaways

- C++ is an extension of C language.
- It is a powerful, efficient and general-purpose programming language.
- It supports both Procedural and Object-Oriented Programming.
- Every C++ program must have main() function.
- cout is used for output and cin is used for input.
- C++ programs are compiled and executed to produce results.



15. Tokens in C++

Tokens are the smallest individual units in a C++ program.

Remember!
Every C++ program is made up of various tokens.

Token Type	Description	Example	Example in Code
1. Keywords	Reserved words with special meaning.	int, float, if, else, return	`int x = 10;`
2. Identifiers	User defined names for variables, functions, etc.	sum, total, main, value	`int sum = 0;`
3. Constants	Fixed values that do not change.	10, 3.14, 'A', true	`float pi = 3.14;`
4. Operators	Symbols that perform operations.	+, -, *, /, ==, !=, <, >	`int c = a + b;`
5. Separators	Used to separate individual elements.	( ), { }, [ ], ; , :	`if (a > b) { ... }`
6. Literals	Fixed values written directly in code.	10, 2.5, 'A', "Hello"	`char ch = 'A';`

16. Data Types in C++

Choosing the right data type saves memory and improves program performance.

Data types specify the type of data a variable can store.

1. Basic Data Types

- int → Integer numbers (Ex: -10, 0, 25)
- float → Decimal numbers (Ex: 3.14)
- double → More precision decimal numbers
- char → Single character (Ex: 'A', 'b')
- bool → Boolean values (true or false)

Example

```
int   age = 20;
float price = 99.50;
char  grade = 'A';
bool  isPassed = true;
```

2. Derived Data Types

- Array → Collection of similar data types.
- Pointer → Stores address of a variable.
- Reference → Alias of a variable.
- Function → Collection of statements.

Example

```
int  arr[5];
int  *ptr;
int  &ref = age;
void show();
```

3. User Defined Data Types

- Structure → Group of different data types.
- Union → Saves memory by sharing memory.
- Enumeration → List of named integer constants.
- Typedef → Creates new name for existing type.

Example

```
struct Student { int id; };
enum Color { Red, Green, Blue };
typedef unsigned long ulong;
```

NOTE

Quick Summary

→ Tokens are the building blocks of a C++ program.
→ Data types define the kind of data a variable can hold.
→ Proper use of data types makes the program efficient and error-free.

17. Variables in C++

A variable is a named memory location used to store data that can be changed during program execution.

Syntax :

```
data_type  variable_name  =  value;
```

Example :

```
int     age = 20;         // integer variable
float   height = 5.9;     // float variable
char    grade = 'A';      // character variable
bool    isStudent = true; // boolean variable
```

Important Points

- A variable must be declared before it is used.
- It has a specific data type.
- It occupies memory.
- It can store different values during execution.

Rules for Variable Names

- Must start with a letter or underscore (_)
- Can contain letters, digits and underscore
- Cannot start with a digit
- C++ is case sensitive (age and Age are different)
- Cannot use reserved keywords like int, float, if, else, etc.

Example Program :

```
#include <iostream>
using namespace std;
int main() {
   int    age = 20;
   float  height = 5.9;
   char   grade = 'A';
   bool   isStudent = true;

   cout << "Age: " << age << endl;
   cout << "Height: " << height << endl;
   cout << "Grade: " << grade << endl;
   cout << "Is Student: " << isStudent << endl;

   return 0;
}
```

Output :

```
Age: 20
Height: 5.9
Grade: A
Is Student: 1
```

18. Constants in C++

A constant is a fixed value that cannot be changed during program execution.

Types of Constants

Type	Example	Explanation
Integer Constant	10, -25, 1000	Whole numbers
Floating Constant	3.14, -0.25, 2.5e3	Decimal numbers
Character Constant	'A', 'b', '7'	Single character
String Literal	"Hello", "C++"	Sequence of characters
Boolean Constant	true, false	Logical values

Example :

```
const  int     MAX = 100;          // integer constant
const  float   PI = 3.1416;        // float constant
const  char    grade = 'A';        // char constant
const  string  msg = "Hello C++"; // string constant
const  bool    flag = true;        // boolean constant
```

Note :

We use 'const' keyword to declare constants.
Once a value is assigned, it cannot be changed.

19. Operators in C++

Operators are symbols that perform operations on variables and values.

Remember!
Operators make it possible to perform calculations, comparisons and logical operations in a program.

Type of Operator	Operators	Example	Meaning	Example in Code
1. Arithmetic Operators	+ - * / % ++ --	a + b a % b	Perform mathematical operations	int a = 10, b = 3; int c = a + b; // 13

2. Relational Operators	== != > < >= <=	a == b
		a > b
			Compare two values and return true or false	if (a > b) {
				cout << "a is greater";
				}

3. Logical Operators	&& || !	(a > b) &&
		(b > 0)
			Combine conditions or reverse a condition	if (a > 0 && b > 0) {
				cout << "Both are positive";
				}

4. Assignment Operators	= += -= *= /= %=	a += 5	Assign value to a variable	int a = 10; a += 5; // a = a + 5
5. Bitwise Operators	& | ^ ~ << >>	a & b a << 1	Perform bit-level operations	int x = 5, y = 3; int z = x & y; // 1
6. Conditional Operator	?:	(a > b) ? a : b	Ternary operator (if-else in one line)	int max = (a > b) ? a : b;
7. Special Operators	. -> sizeof typeid	obj.name ptr->data	Access members, size of data, type info	sizeof(int); ptr->value;

20. Operator Precedence

Operators are evaluated in a specific order. Higher precedence operators are evaluated first.

Precedence		Operators	Associativity
High	1	(), [], ->, .	Left to Right
	2	++, -- (postfix)	Left to Right
	3	++, -- (prefix), +, -, !, ~, sizeof	Right to Left
	4	*, /, %	Left to Right
	5	+, -	Left to Right
	6	<<, >>	Left to Right
	7	<, <=, >, >=	Left to Right
	8	==, !=	Left to Right
	9	&	Left to Right
	10	^	Left to Right
	11	|	Left to Right
	12	&&	Left to Right
	13	||	Left to Right
	14	?:	Right to Left
Low	15	=, +=, -=, *=, /=, %=	Right to Left

21. Example Program Using Operators

```cpp
#include <iostream>
using namespace std;
int main() {
    int a = 10, b = 3;
    int sum = a + b;
    int diff = a - b;
    bool check = (a > b) && (b > 0);
    cout << "Sum: " << sum << endl;
    cout << "Difference: " << diff << endl;
    cout << "Check: " << (check ? "True" : "False") << endl;
    return 0;
}
```

Output:
Sum: 13
Difference: 7
Check: True

Key Takeaway

- Operators are essential to perform operations in C++.
- Understanding precedence helps in writing correct expressions.
- Use parentheses () to change the default precedence.

22. Decision Making in C++ (if Statement)

Decision making statements allow a program to take different actions based on conditions.

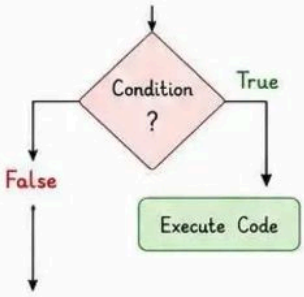
Remember!

if statement is used to execute a block of code only when a condition is true.

Types of if Statements

1. if Statement

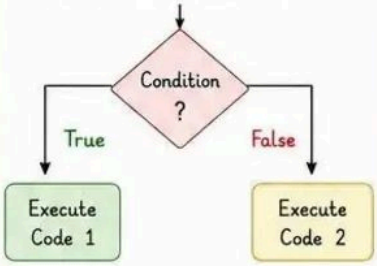
Executes a block of code if the condition is true.




```
if (condition) {
    // code
}
```

2. if – else Statement

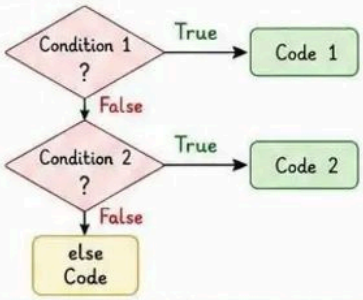
Executes one block if the condition is true, otherwise executes another block.




```
if (condition) {
    // code 1
} else {
    // code 2
}
```

3. if – else if – else Statement

Checks multiple conditions and executes the first true block. If none are true, executes else block.




```
if (condition1) {
    // code 1
} else if (condition2) {
    // code 2
} else {
    // else code
}
```

23. Example Program

Program to find the largest number among two numbers using if–else statement.

```
#include <iostream>
using namespace std;
int main() {
   int a, b;
   cout << "Enter two numbers: ";
   cin >> a >> b;
   if (a > b) {
      cout << a << " is greater.";
   } else {
      cout << b << " is greater.";
   }
   return 0;
}
```

Sample Run:

Enter two numbers: 25 18

25 is greater.

24. Nested if Statement

An if statement inside another if or else block.

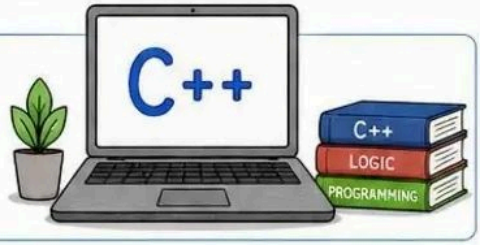
```
if (condition1) {
    if (condition2) {
        // code when both true
    } else {
        // code when condition2 false
    }
} else {
    // code when condition1 false
}
```

Use When:

You need to check multiple conditions in a hierarchical manner.

Key Takeaways

- Decision making allows the program to act based on conditions.
- if, if–else and if–else if–else are the main decision making statements.
- Use nested if when one condition depends on another.
- These statements make programs smarter and more flexible.



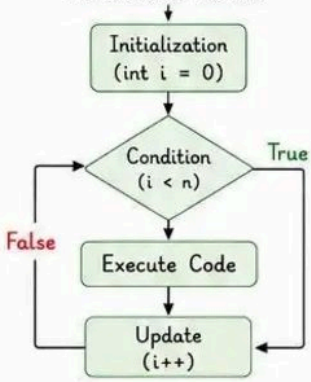
25. Loops in C++

Loops are used to repeat a block of code multiple times until a condition is met.

Remember!
Loops save time and effort by automating repetition.

1. for Loop

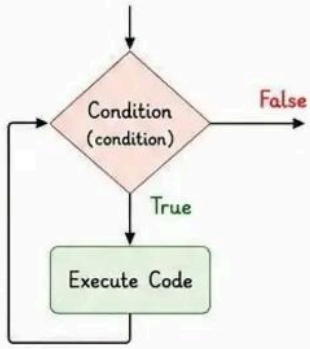
Used when the number of iterations is known.




```
for (initialization; condition; update) {
    // code to be executed
}
```

2. while Loop

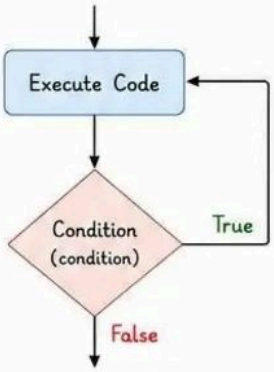
Used when the number of iterations is not known.




```
while (condition) {
    // code to be executed
}
```

3. do - while Loop

Similar to while loop, but executes the code block at least once.




```
do {
    // code to be executed
} while (condition);
```

26. Example: for Loop

Program to print numbers from 1 to 5 using for loop.

```
#include <iostream>
using namespace std;
int main() {
  for (int i = 1; i <= 5; i++) {
      cout << i << " ";
  }
  return 0;
}
```

Output:
1 2 3 4 5

How it works?

- Initialization: i = 1 (starts the loop)
- Condition: i <= 5 (check before each iteration)
- Execute Code: print i
- Update: i++ (increase i by 1)
- Repeat until condition becomes false

27. Example: while Loop

Program to calculate the sum of first 5 natural numbers.

```
#include <iostream>
using namespace std;
int main() {
  int i = 1, sum = 0;
  while (i <= 5) {
      sum = sum + i;
      i++;
  }
  cout << "Sum = " << sum;
  return 0;
}
```

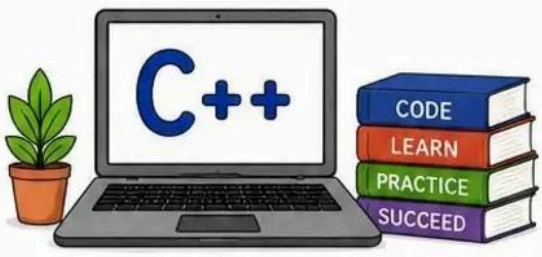
Output:
Sum = 15

How it works?

- Start with i = 1, sum = 0
- Check condition (i <= 5)
- Add i to sum, then increase i
- Repeat until i becomes 6 (condition false)
- Final sum = 15

28. Key Takeaways

- Loops help in repeating tasks and reduce code length.
- Use for loop when the number of iterations is known.
- Use while loop when the number of iterations is unknown.
- Use do-while loop when the code needs to run at least once.
- Loops are powerful tools for efficient programming.



29. Functions in C++

Functions are used to divide a program into smaller blocks to improve readability, reusability and maintenance.

Remember!
Functions help you write once and use many times.

Anatomy of a Function

```
return_type  function_name(parameters) {
    // function body
    return value;
}
```

- **Return Type** — Type of value returned by function.
- **Function Name** — Name of the function.
- **Parameters** — Input values passed to the function.
- **Function Body** — Set of statements to perform a specific task.

Types of Functions

Type	Description
1. No Argument, No Return	No input, no output.
2. With Argument, No Return	Takes input, no output.
3. No Argument, With Return	No input, returns output.
4. With Argument, With Return	Takes input and returns output.

30. Example Programs

1. No Argument, No Return

```
#include <iostream>
using namespace std;
void greet() {
   cout << "Hello C++!";
}
int main() {
   greet();
   return 0;
}
```

Output:
Hello C++!

2. With Argument, No Return

```
#include <iostream>
using namespace std;
void printNum(int n) {
   cout << "Number: "
        << n;
}
int main() {
   printNum(25);
   return 0;
}
```

Output:
Number: 25

3. No Argument, With Return

```
#include <iostream>
using namespace std;
int getNumber() {
   return 100;
}
int main() {
   int x = getNumber();
   cout << "Value: "
        << x;
   return 0;
}
```

Output:
Value: 100

4. With Argument, With Return

```
#include <iostream>
using namespace std;
int add(int a, int b) {
   return a + b;
}
int main() {
   int sum = add(10, 20);
   cout << "Sum: "
        << sum;
   return 0;
}
```

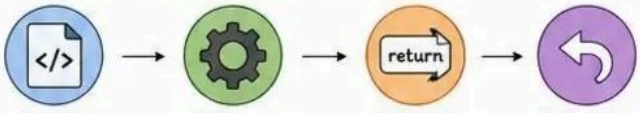
Output:
Sum: 30

Benefits of Using Functions

- Improves readability of code
- Reduces code duplication
- Makes the program modular
- Easy to test and debug
- Enhances reusability



31. How Function Call Works



1. **Call** — Function is called from main() or another function.
2. **Execute** — Control goes to the function and executes the code.
3. **Return** — After execution, control returns to the calling point.
4. **Continue** — Program continues from the next statement after the call.

Quick Summary

- Functions make code organized and easy to manage.
- A function may or may not take arguments and may or may not return a value.
- Using functions helps in building large and complex programs efficiently.



32. Arrays in C++

Arrays are used to store multiple values of the same type under a single name.

Remember!
Arrays help you store and manage lists of data efficiently.

1. One-Dimensional Array

```
int arr[5];
```

Index ->	0	1	2	3	4
Value ->	10	20	30	40	50

```
int arr[5];     // Declaration
arr[0] = 10;
arr[1] = 20;
...
arr[4] = 50;    // Last element
```

Note:
- Index starts from 0
- Size is fixed and must be a positive integer

2. Two-Dimensional Array

```
int mat[2][3];
```

	0	1	2
0	1	2	3
1	4	5	6

```
int mat[2][3] = {
   {1, 2, 3},
   {4, 5, 6}
};
```

Note:
- First index = row
- Second index = column

3. Common Operations

Operation	Example / Description
Declaration	`int arr[5];` Creates an array of 5 integers.
Access	`arr[2] = 30;` Access third element.
Traversal	Use loops to access all elements.
Size	Size is fixed at declaration time.
Initialization	`int arr[3] = {10, 20, 30};` Initialize while declaring.

33. Example Programs

1. Print Elements of Array

```
#include <iostream>
using namespace std;
int main() {
   int arr[5] = {10, 20, 30, 40, 50};
   for (int i = 0; i < 5; i++)
      cout << arr[i] << " ";
   return 0;
}
```

Output:
10 20 30 40 50

2. Sum of Array Elements

```
#include <iostream>
using namespace std;
int main() {
   int arr[5] = {2, 4, 6, 8, 10};
   int sum = 0;
   for (int i = 0; i < 5; i++)
      sum += arr[i];
   cout << "Sum = " << sum;
   return 0;
}
```

Output:
Sum = 30

3. 2D Array Example

```
#include <iostream>
using namespace std;
int main() {
  int mat[2][3] = {{1, 2, 3}, {4, 5, 6}};
  for (int i = 0; i < 2; i++) {
     for (int j = 0; j < 3; j++)
        cout << mat[i][j] << " ";
     cout << endl;
  }
  return 0;
}
```

Output:
1 2 3
4 5 6

34. Key Takeaways

- Arrays store multiple values of the same type.
- Indexing starts from 0.
- Size of array is fixed once declared.
- Use loops to efficiently access and process array elements.
- 2D arrays are useful for representing tables, matrices and grids.

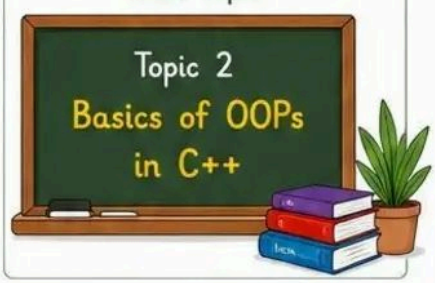
Memory Representation

```
int arr[5] = {10, 20, 30, 40, 50};
```

Index :	0	1	2	3	4
Address:	1000	1004	1008	1012	1016
Value :	10	20	30	40	50

Tip: Each integer usually takes 4 bytes of memory.

Next Topic



Keep Practicing, Keep Learning, Keep Growing!
Best of Luck for your C++ Journey!